Lab no: 21 **Date: 2082/03/30**

Title: Implementation of Delegate.

Theory:

A delegate is an object that refers to a method or it is a variable of the reference type that can hold a reference of the methods.

There are three types of delegates in C#. They are:

1. Single Delegate
2. Multi Delegate
3. Genric Delegate
    a. Func Delegate
    b. Action Delegate
    c. Predict Delegae

Source Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace _21a__Delegatetesall
{
   //implementation of Delegate
   public delegate void Calculations(int a , int b);
   internal class Program
   {
      public static void Addition(int a, int b)
      {
         int result = a + b;
         Console.WriteLine("Addition result is :{0}", result);
      }

      public static void Subtraction (int a , int b)
      {
         int result = a - b;
         Console.WriteLine("Subraction result is :{0}", result);
      }

      public static void Multiplication(int a, int b)
      {
         int result = a * b;
         Console.WriteLine("Multiplication result is :{0}", result);
      }

      public static void Division(int a, int b)
      {
```

```
        int result = a / b;
        Console.WriteLine("Division  result is :{0}", result);
    }


    static void Main(string[]  args)
    {
        Calculations  obj =new Calculations(Program.Addition);
        obj.Invoke(20, 10);
        Calculations  obj1 = new Calculations(Program.Subtraction);
        obj1.Invoke(20, 10);
        Calculations  obj2 = new Calculations(Program.Multiplication);
        obj2.Invoke(2,2);
        Calculations  obj3 = new Calculations(Program.Division);
        obj3.Invoke(10,5);
        Console.ReadLine();
    }

  }
}
```

Output:

```
Microsoft Visual Studio Debu

Addition result is :30
Subraction result is :10
Multiplication result is :4
Division result is :2


C:\Users\Acer\source\repos\_21a _Delegatetesall\_21a _Delegatetesall\bin\Debug\_21a _Delegatetesall.exe (process 1784) e
xited with code 0 (0x0).
Press any key to close this window . . .
```

Lab no: 22 **Date: 2082/03/32**

Title: CLI commands in Shell.

Output:

1. PS C:\Users\Acer> dotnet –info

Displays detailed information about the .NET SDK and runtime environment installed on your machine, including version, OS, and system architecture. Useful for troubleshooting.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Acer> dotnet --info
.NET SDK:
 Version:           9.0.301
 Commit:            a596cd22e2
 Workload version:  9.0.300-manifests.e42facbb
 MSBuild version:   17.14.5+edd3bbf37

Runtime Environment:
 OS Name:     Windows
 OS Version:  10.0.26100
 OS Platform: Windows
 RID:         win-x64
 Base Path:   C:\Program Files\dotnet\sdk\9.0.301\

.NET workloads installed:
There are no installed workloads to display.
Configured to use loose manifests when installing new manifests.

Host:
  Version:      9.0.6
  Architecture: x64
  Commit:       3875b54e7b

.NET SDKs installed:
  9.0.301 [C:\Program Files\dotnet\sdk]

.NET runtimes installed:
  Microsoft.AspNetCore.App 8.0.17 [C:\Program Files\dotnet\shared\Microsoft.AspNetCore.App]
  Microsoft.AspNetCore.App 9.0.6 [C:\Program Files\dotnet\shared\Microsoft.AspNetCore.App]
  Microsoft.NETCore.App 8.0.8 [C:\Program Files\dotnet\shared\Microsoft.NETCore.App]
  Microsoft.NETCore.App 8.0.17 [C:\Program Files\dotnet\shared\Microsoft.NETCore.App]
  Microsoft.NETCore.App 9.0.6 [C:\Program Files\dotnet\shared\Microsoft.NETCore.App]
  Microsoft.WindowsDesktop.App 8.0.8 [C:\Program Files\dotnet\shared\Microsoft.WindowsDesktop.App]
  Microsoft.WindowsDesktop.App 8.0.17 [C:\Program Files\dotnet\shared\Microsoft.WindowsDesktop.App]
  Microsoft.WindowsDesktop.App 9.0.6 [C:\Program Files\dotnet\shared\Microsoft.WindowsDesktop.App]

Other architectures found:
  x86   [C:\Program Files (x86)\dotnet]
    registered at [HKLM\SOFTWARE\dotnet\Setup\InstalledVersions\x86\InstallLocation]

Environment variables:
  Not set

global.json file:
  Not found

Learn more:
  https://aka.ms/dotnet/info

Download .NET:
  https://aka.ms/dotnet/download
```

2. PS C:\Users\Acer> dotnet new list

Lists all available project templates (like console app, class library, web API, etc.) that you can use to create new .NET projects using dotnet new.

```
PS C:\Users\Acer> dotnet new  list
These templates matched your input:

Template Name       Short Name                  Language    Tags
------------------  --------------------------  ----------  ---------------------------------
API Controller      apicontroller               [C#]        Web/ASP.NET
ASP.NET Core Empty  web                         [C#],F#     Web/Empty
ASP.NET Core gR...  grpc                        [C#]        Web/gRPC/API/Service
ASP.NET Core We...  webapi                      [C#],F#     Web/Web API/API/Service
ASP.NET Core We...  webapiaot                   [C#]        Web/Web API/API/Service
ASP.NET Core We...  mvc                         [C#],F#     Web/MVC
ASP.NET Core We...  webapp,razor                [C#]        Web/MVC/Razor Pages
Blazor Web App      blazor                      [C#]        Web/Blazor/WebAssembly
Blazor WebAssem...  blazorwasm                  [C#]        Web/Blazor/WebAssembly/PWA
Class Library       classlib                    [C#],F#,VB  Common/Library
Console App         console                     [C#],F#,VB  Common/Console
dotnet gitignor...  gitignore,.gitignore                    Config
Dotnet local to...  tool-manifest                           Config
EditorConfig file   editorconfig,.editorconfig              Config
global.json file    globaljson,global.json                  Config
MSBuild Directo...  buildprops                              MSBuild/props
MSBuild Directo...  buildtargets                            MSBuild/props
MSBuild Directo...  packagesprops                           MSBuild/packages/props/CPM
MSTest Playwrig...  mstest-playwright           [C#]        Test/MSTest/Playwright/Desktop/Web
MSTest Test Class   mstest-class                [C#],F#,VB  Test/MSTest
MSTest Test Pro...  mstest                      [C#],F#,VB  Test/MSTest/Desktop/Web
MVC Controller      mvccontroller               [C#]        Web/ASP.NET
MVC ViewImports     viewimports                 [C#]        Web/ASP.NET
MVC ViewStart       viewstart                   [C#]        Web/ASP.NET
NuGet Config        nugetconfig,nuget.config                Config
NUnit 3 Test Item   nunit-test                  [C#],F#,VB  Test/NUnit
NUnit 3 Test Pr...  nunit                       [C#],F#,VB  Test/NUnit/Desktop/Web
NUnit Playwrigh...  nunit-playwright            [C#]        Test/NUnit/Playwright/Desktop/Web
Protocol Buffer...  proto                                   Web/gRPC
Razor Class Lib...  razorclasslib               [C#]        Web/Razor/Library
Razor Component     razorcomponent              [C#]        Web/ASP.NET
Razor Page          page                        [C#]        Web/ASP.NET
Razor View          view                        [C#]        Web/ASP.NET
Solution File       sln,solution                            Solution
Web Config          webconfig                               Config
Windows Forms App   winforms                    [C#],VB     Common/WinForms
Windows Forms C...  winformslib                 [C#],VB     Common/WinForms
Windows Forms C...  winformscontrollib          [C#],VB     Common/WinForms
Worker Service      worker                      [C#],F#     Common/Worker/Web
WPF Application     wpf                         [C#],VB     Common/WPF
WPF Class Library   wpflib                      [C#],VB     Common/WPF
WPF Custom Cont...  wpfcustomcontrollib         [C#],VB     Common/WPF
WPF User Contro...  wpfusercontrollib           [C#],VB     Common/WPF
xUnit Test Project  xunit                       [C#],F#,VB  Test/xUnit/Desktop/Web

PS C:\Users\Acer> |
```

3. PS C:\Users\Acer> dotnet new console -o DemoApp

Adds the NuGet package named DemoApp to the current project. This command is used to include external libraries in your project for extended functionality

```
PS C:\Users\Acer> dotnet new console -o DemoApp
The template "Console App" was created successfully.

Processing post-creation actions...
Restoring C:\Users\Acer\DemoApp\DemoApp.csproj:
Restore succeeded.


PS C:\Users\Acer>
```

4. **PS C:\Users\Acer> cd DemoApp**
5. **PS C:\Users\Acer\DemoApp> dotnet add package Dopper**

```
PS C:\Users\Acer> cd DemoApp
PS C:\Users\Acer\DemoApp> dotnet add package Dopper

Build succeeded in 0.5s
info : X.509 certificate chain validation will use the default trust store selected by .NET for code signing.
info : X.509 certificate chain validation will use the default trust store selected by .NET for timestamping.
info : Adding PackageReference for package 'Dopper' into project 'C:\Users\Acer\DemoApp\DemoApp.csproj'.
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/dopper/index.json
info :   NotFound https://api.nuget.org/v3/registration5-gz-semver2/dopper/index.json 308ms
info :   GET https://api.nuget.org/v3/registration5-gz-semver2/dopper/index.json
info :   NotFound https://api.nuget.org/v3/registration5-gz-semver2/dopper/index.json 446ms
error: There are no versions available for the package 'Dopper'.
PS C:\Users\Acer\DemoApp>
```

6. **PS C:\Users\Acer\DemoApp> dotnet restore**
   Downloads and installs any missing NuGet packages required by the project. It ensures all dependencies are available before building the project.

```
PS C:\Users\Acer\DemoApp> dotnet restore
Restore complete (0.3s)

Build succeeded in 0.6s
PS C:\Users\Acer\DemoApp>
```

7. **PS C:\Users\Acer\DemoApp> dotnet build**
   Compiles the application's source code into executable files or libraries. It checks for errors and prepares the app for running.

8. **PS C:\Users\Acer\DemoApp> dotnet clean**
   Removes all previously compiled output (like DLLs and EXEs) from the build folder. This is useful for cleaning up and rebuilding from scratch.

9. **PS C:\Users\Acer\DemoApp> dotnet run**

Builds and runs the application in a single step. Ideal for quickly testing or running your app during development.

```
PS C:\Users\Acer\DemoApp> dotnet build
Restore complete (0.3s)
  DemoApp succeeded (3.7s) → bin\Debug\net9.0\DemoApp.dll

Build succeeded in 4.4s
PS C:\Users\Acer\DemoApp> dotnet clean

Build succeeded in 0.4s
PS C:\Users\Acer\DemoApp> dotnet run
Hello, World!
PS C:\Users\Acer\DemoApp>
```

Lab no: 23 **Date: 2082/04/13**

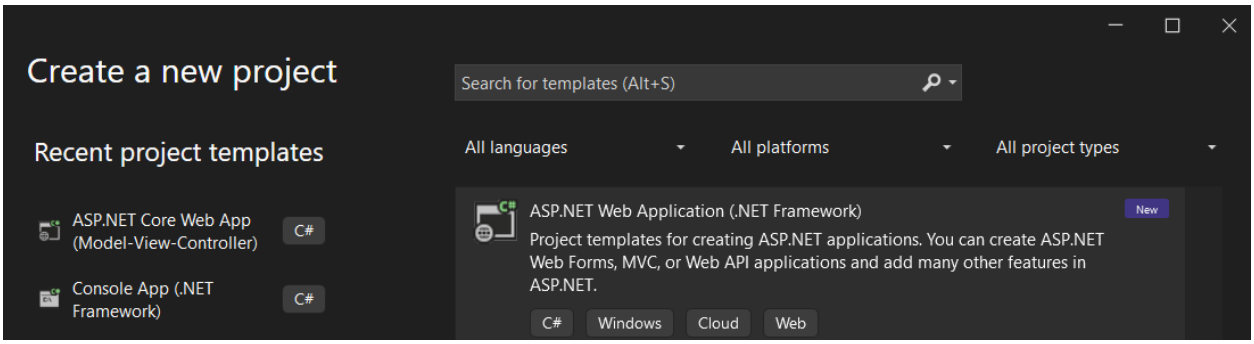
Title: Implementation of Razor View Engine(MVC→model, view, controller).

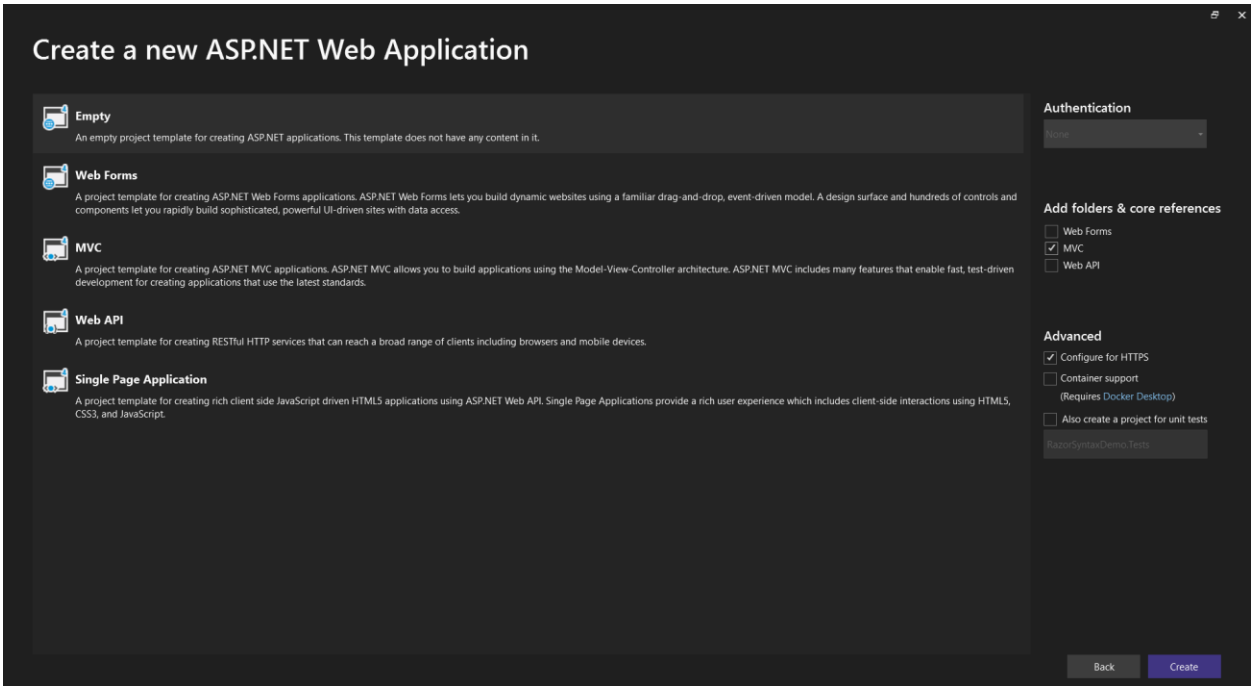
Theory:

Razor View Engine is a view engine, the default view engine for the ASP.NET Core application. The Razor file extension is "C#html" for the C# language. A view provides the UI of the application to the user which is used to display content of the application and also to accept the user input.

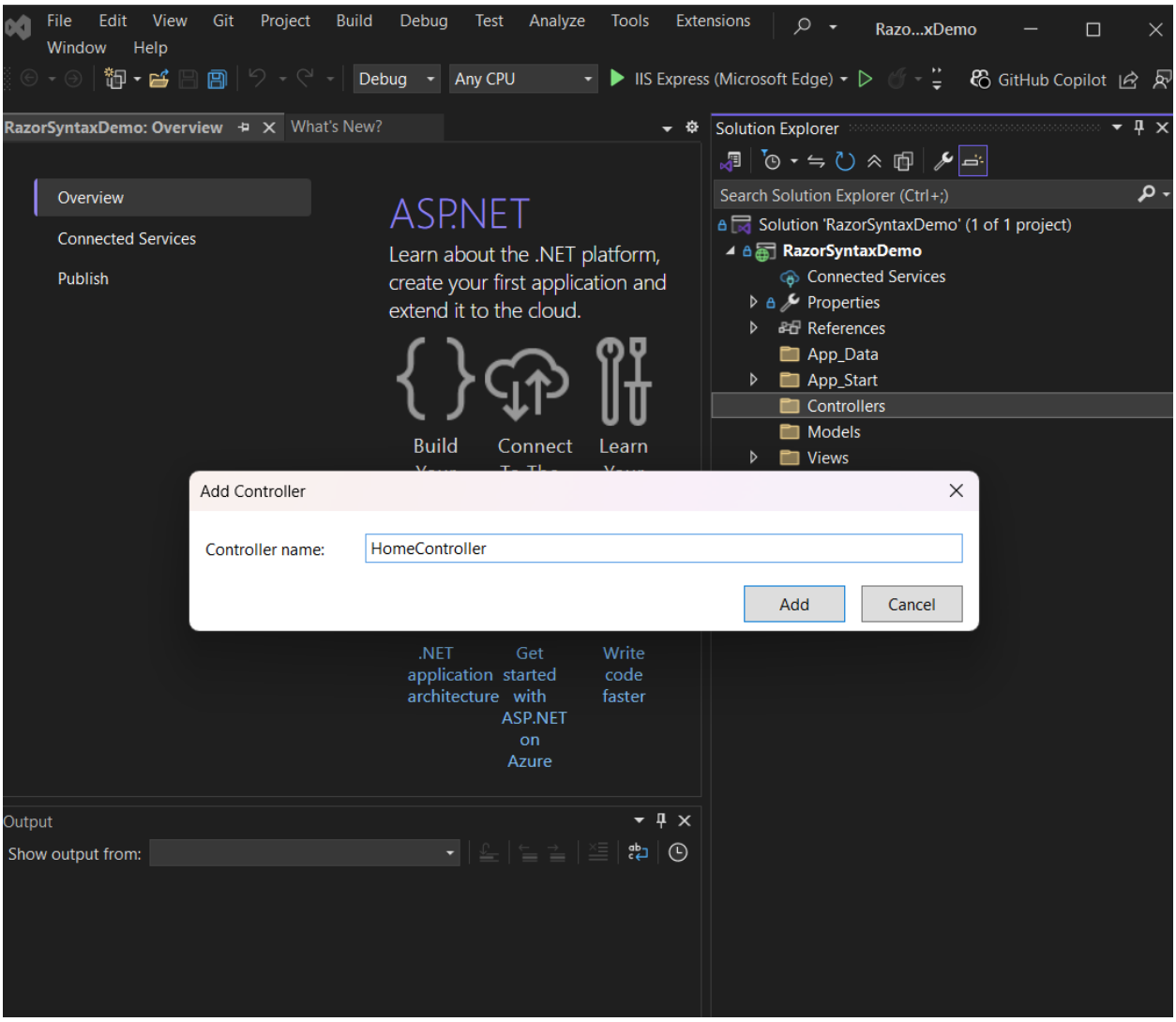
Output:

Step 1: Visual Studio→Create New Project→ASP.NET Web Application(.Net Framework)→Project Name(Razor Syntax Demo)→OK→Yes→Empty→ ✓MVC→OK





Step 2:RazorSyntaxdemo→App_Start →Controller(Right Click)→ Add→ Controller →MVC5 controller-empty→Controller Name(HomeController)→Add

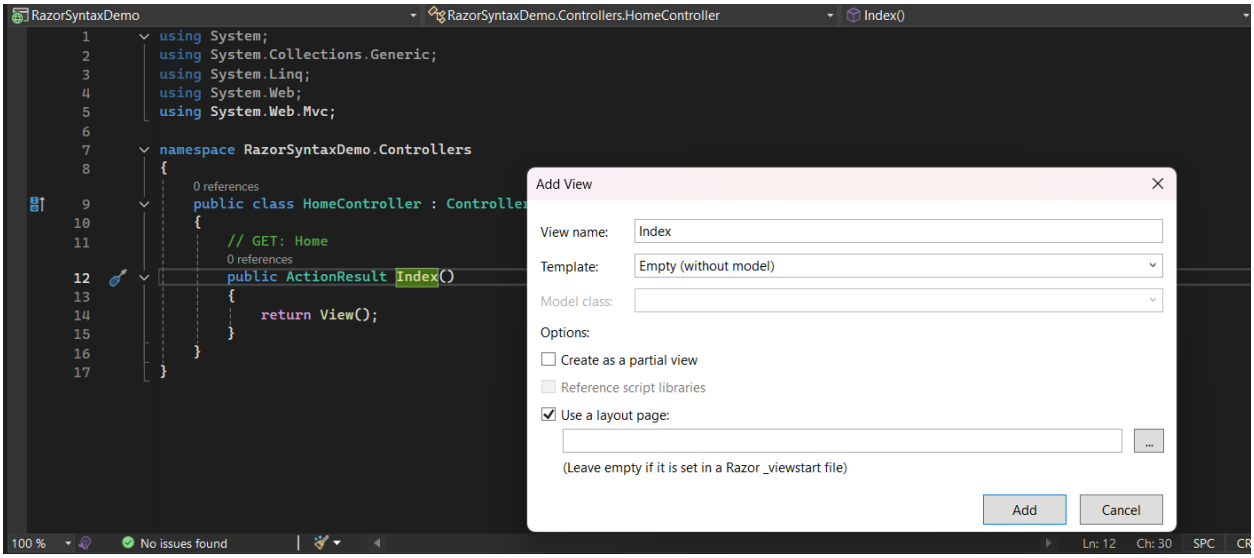


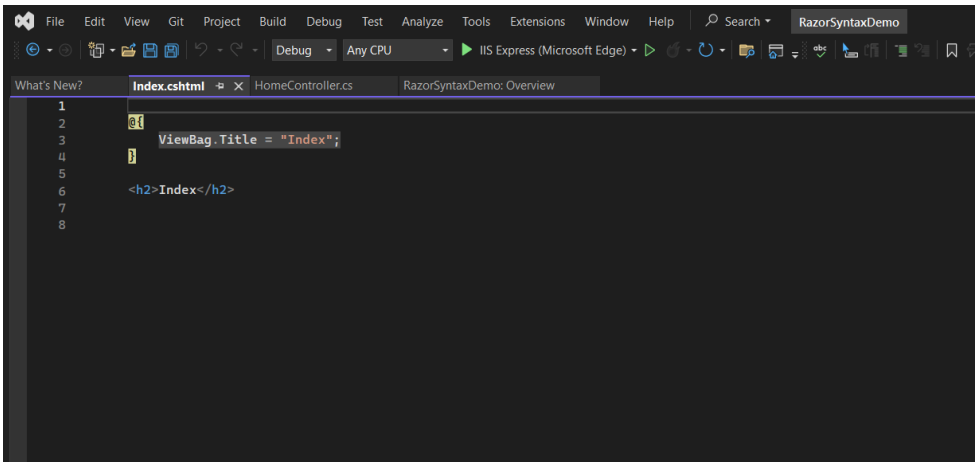
Step 3:Solution Explorer→View →Home→Right Click →Add →View →MVCS View →Add→

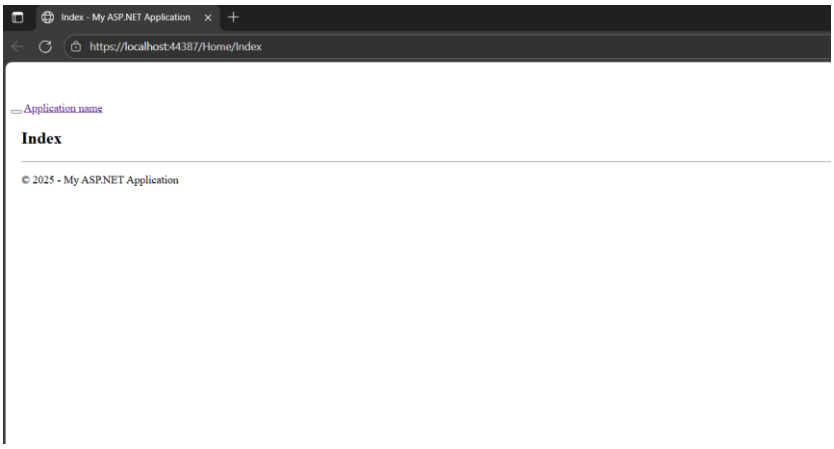
View Name(Index)

OR

Solution(Index)→Right Click→Add →View→Name(Index)→OK







Step 4: Today Date and Time is @DatetTime.Now

Step 5:

```
@{
    ViewBag.Title = "Index";
}

<h2>Index</h2>
<p>Today Date and Time is @DateTime.Now</p>

<table border="1">
    <tr>
        <th width="100px">Name</th>
        <th width="100px">Value</th>
    </tr>
    <tr>
        <td>Day</td>
        <td>@DateTime.Now.Day</td>
    </tr>
    <tr>
        <td>Hour</td>
        <td>@DateTime.Now.Hour</td>
    </tr>
    <tr>
        <td>Minute</td>
        <td>@DateTime.Now.Minute</td>
    </tr>
    <tr>
        <td>Second</td>
        <td>@DateTime.Now.Second</td>
    </tr>

</table>
```

Index - My ASP.NET Application

https://localhost:44387/Home/Index

Application name

Index

Today Date and Time is 7/31/2025 7:45:36 AM

Name	Value
Day	31
Hour	7
Minute	45
Second	36

© 2025 - My ASP.NET Application

Lab no: 24 **Date: 2082/04/15**
Title: Implementation of Action Methods(controller) in ASP.Net MVC.

Theory:

This code defines a HomeController in an ASP.NET Core MVC application. It handles web requests and returns corresponding views like Index, Privacy, and Error. The controller uses dependency injection to get an ILogger for logging. The Error() action displays error details using the ErrorViewModel, while the [ResponseCache] attribute ensures the error page is not cached. Overall, it demonstrates basic controller structure, action methods, and error handling in MVC.

Output:

HomeController.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;

namespace lab24actioncontroller.Controllers
{
    public class HomeController : Controller
    {
        // GET: Home
        public ActionResult Index()
        {
            return View();
        }
        public string Show()
        {
            return "This is the action method";
        }
        public ActionResult AboutUs()
        {
            return View();
        }
        public int StudentID(int id)
        {
            return id;
        }
    }
}
```

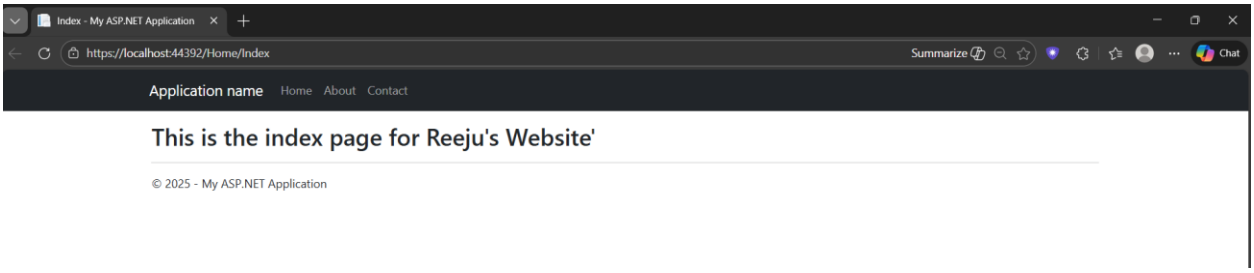
Index.cshtml:

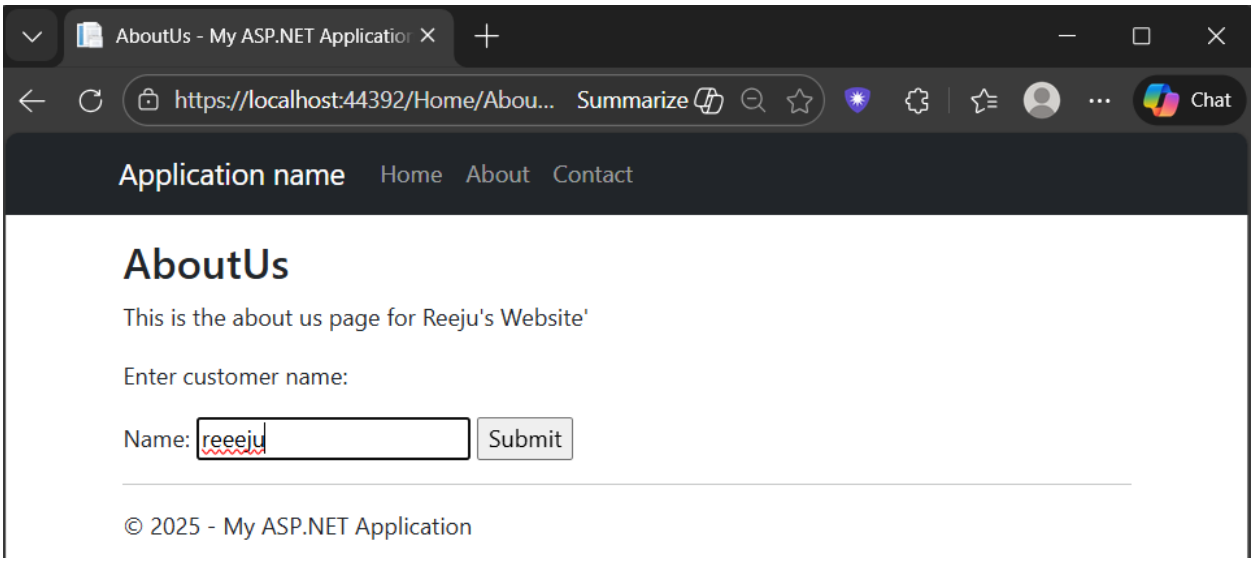
```
@{
    ViewBag.Title = "Index";
}
<h2>This is the index page for Reeju's Website'</h2>
```

AboutUs.cshtml:

```
@{
```

```
    ViewBag.Title = "AboutUs";
}

<h2>AboutUs</h2>
<p>This is the about us page for Reeju's Website'</p>
<p>Enter customer name:</p>
<form method="post">
    Name:
    <input asp-for="Customer! .Name" />
    <input type="submit" />
</form>
```





Lab no: 25 **Date: 2082/04/16**
Title: Implementation of ViewData in ASP.NET MVC. using System.

Theory:

ViewData in ASP.NET MVC is a dictionary object (key–value pair) used to pass data from a controller to a view within the same HTTP request. It belongs to the System.Web.Mvc namespace and stores data as objects, so typecasting is required when retrieving values in the view. It is useful for transferring small, temporary data such as messages or lists but is not type-safe and loses data after the request ends.

Controller:

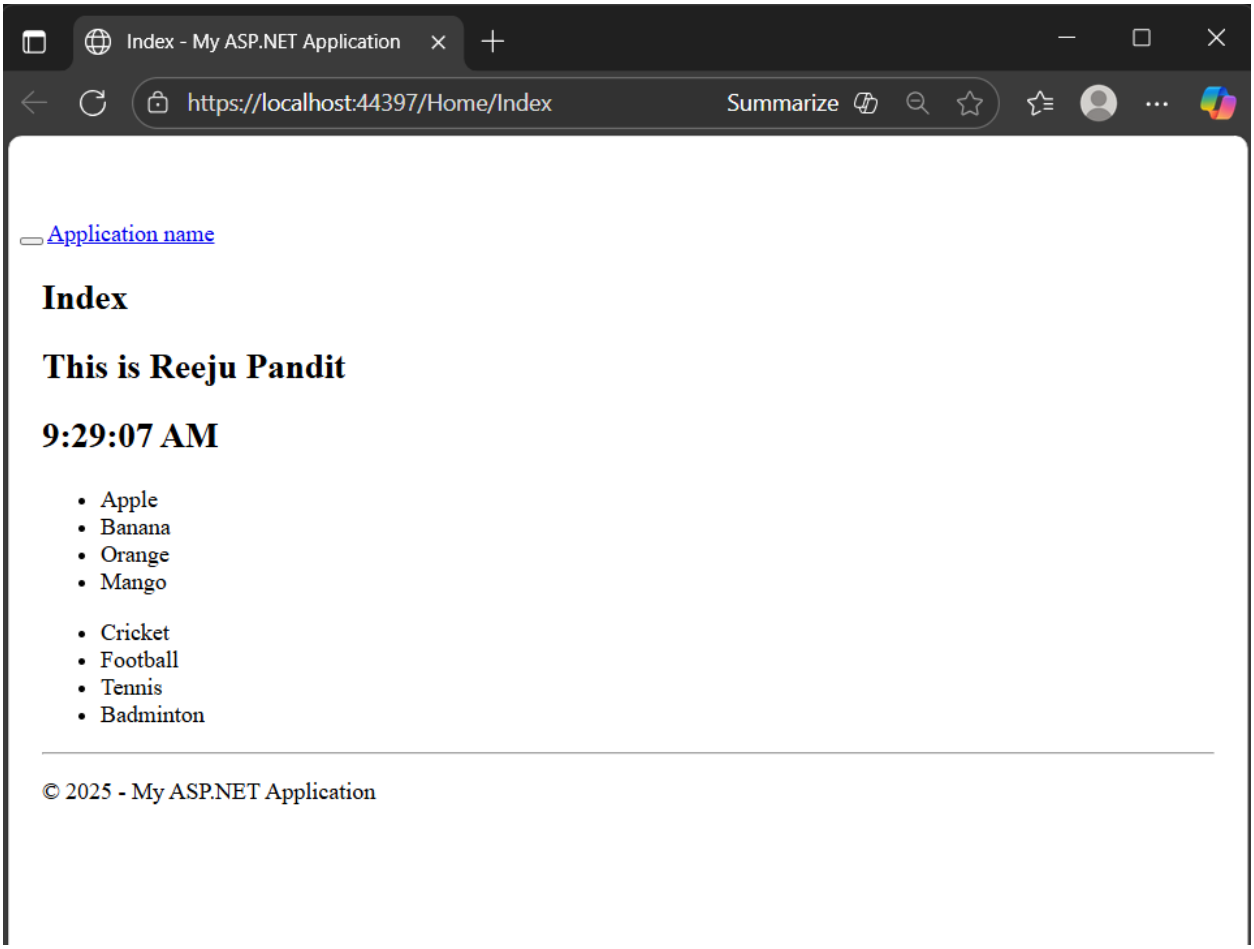
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;

namespace Lab25_View.Controllers
{
    public class HomeController : Controller
    {
        // GET: Home
        public ActionResult Index()
        {
            ViewData["Message"] = "This is Reeju Pandit";
            ViewData["CurrentTime"] = DateTime.Now.ToLongTimeString();
            string[] Fruits = { "Apple", "Banana", "Orange", "Mango" };
            ViewData["FruitsArray"] = Fruits;
            ViewData["SportsList"] = new List<string> { "Cricket", "Football",
"Tennis", "Badminton" };
            return View();
        }
    }
}
```

Index:

```
@{
    ViewBag.Title = "Index";
}

<h2>Index</h2>
<h2>@ViewData["Message"]</h2>
<h2>@ViewData["CurrentTime"]</h2>

<ul>
    @{
        foreach (string i in (string[])ViewData["FruitsArray"])
        {
            <li>@i</li>
        }
    }
```

```
</ul>

<ul>
    @{
        foreach (string i in (List<string>)ViewData["SportsList"])
        {
            <li>@i</li>
        }
    }
</ul>
```

Output:



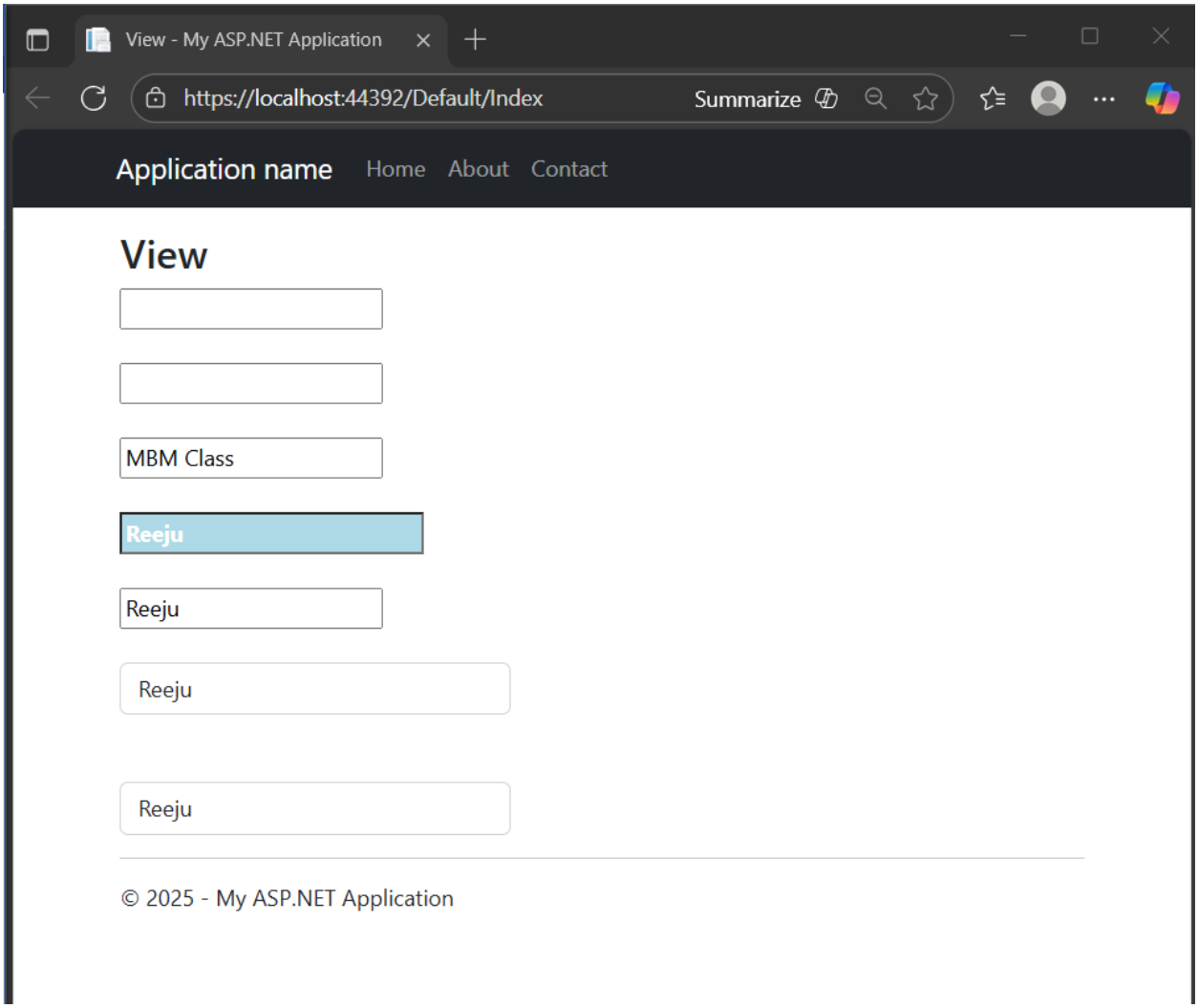
Lab no: 26 **Date: 2082/04/19**
Title: Implementation of html helper method in ASP.NET MVC

Theory:
Html helper methods in ASP.NET MVC are server-side helpers that render HTML controls in Razor views. @Html.TextBox(...) generates an <input type="text"> element for you and lets you supply name, value and HTML attributes (via an anonymous object). Helpers keep markup consistent, support model binding, and make it easy to attach attributes like class, style, or readonly

Controller:
```
Imports System.Web.Mvc

Namespace Controllers
    Public Class DefaultController
        Inherits Controller

        ' GET: Default
        Function Index() As ActionResult
            Return View()
        End Function
    End Class
End Namespace
```
Index:
```
@Code
    ViewData("Title") = "View"
End Code

<h2>View</h2>
<input type="text" name="fullName" id="fullName" value=" " />
<br />
<br />

@Html.TextBox("fullName")
<br />
<br />

@Html.TextBox("fullName", "MBM Class")
<br />
<br />

@Html.TextBox("fullName", "Reeju", New With {
                .style = "background-color:lightblue; color:white; font-weight:
bold;",
                .title = "Please enter your full name"
            })
<br />
<br />

@Html.TextBox("fullName", "Reeju", New With {.class = "TextBoxStyle"})
<br />
<br />

@Html.TextBox("fullName", "Reeju", New With {.class = "form-control"})
<br />
<br />
```

```
@Html.TextBox("fullName", "Reeju", New With {.class = "form-control", .[readonly] =
"readonly"})
```

Output:



Lab no: 27 **Date: 2082/04/20**

Title: Implementation and creation of Form with html helper method in ASP.NET web application MVC.

Theory:

In ASP.NET MVC, HTML Helper methods are used to generate standard HTML form elements like text boxes, dropdowns, and buttons using server-side code. They simplify creating forms that are automatically bound to controller actions and model properties. The most common helper, @Html.BeginForm(), creates a <form> tag, while helpers like @Html.TextBox(), @Html.Password(), and @Html.TextArea() generate input fields. These methods ensure consistent HTML structure, support model binding, and reduce manual coding, making form creation easier, cleaner, and more maintainable in MVC applications.

Controller:

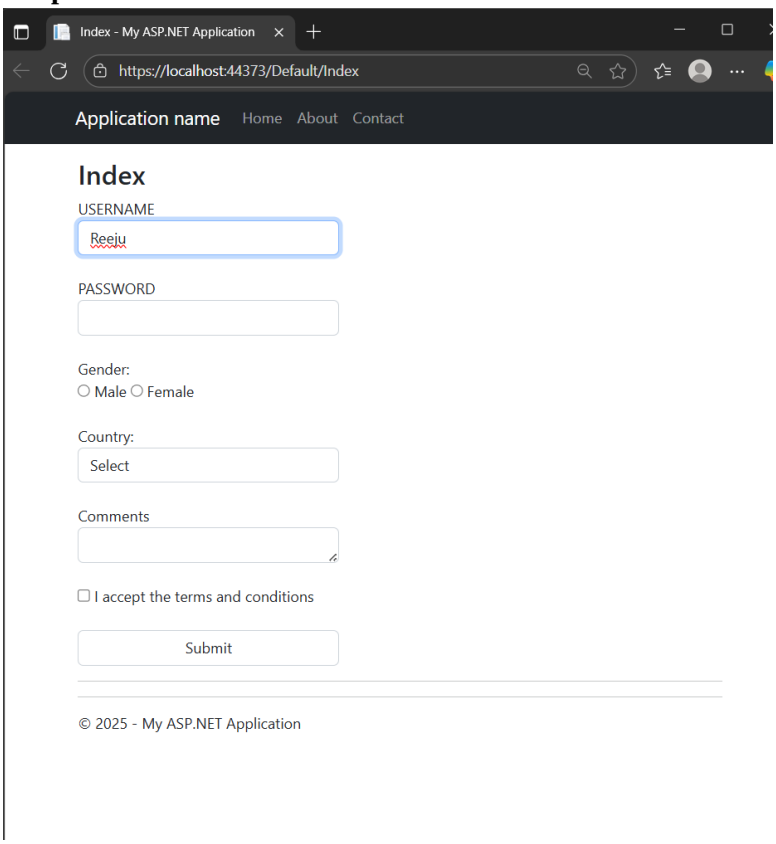
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;

namespace Form.Controllers
{
    public class DefaultController : Controller
    {
        // GET: Default
        public ActionResult Index()
        {
            return View();
        }
    }
}
```

Index:

```
@{
    ViewBag.Title = "Index";
}

<h2>Index</h2>

@using (Html.BeginForm("Index", "Default", FormMethod.Post))
{
    <div>
        @Html.Label("USERNAME")
        @Html.TextBox("Username", "", new { @class = "form-control" })
    </div>
    <br />

    <div>
        @Html.Label("PASSWORD")
        @Html.Password("pass", "", new { @class = "form-control" })
    </div>
    <br />

    <div>
        @Html.Label("Gender:")
        <div>
            @Html.RadioButton("Gender", "Male") @Html.Label("Male")
            @Html.RadioButton("Gender", "Female") @Html.Label("Female")
        </div>
    </div>
    <br />

    <div>
        @Html.Label("Country:")
```

```
        @Html.DropDownList("Countrylist",
            new SelectList(new[] { "Nepal", "India", "Pakistan", "Srilanka", "Bangladesh" }),
            "Select",
            new { @class = "form-control" })
    </div>
    <br />

    <div>
        @Html.Label("Comments")
        @Html.TextArea("CommentsArea", "", 1, 10, new { @class = "form-control" })
    </div>
    <br />

    <div>
        @Html.CheckBox("AcceptTerms")
        @Html.Label("I accept the terms and conditions")
    </div>
    <br />

    <input type="submit" value="Submit" class="form-control" />
}
<hr />

@if (ViewBag.Message != null)
{
    <div>
        <strong>@ViewBag.Message</strong>
        <p>Username: @ViewBag.Username</p>
        <p>Gender: @ViewBag.Gender</p>
        <p>Country: @ViewBag.Country</p>
        <p>Comments: @ViewBag.Comments</p>
        <p>Accepted Terms: @(ViewBag.Accepted ? "Yes" : "No")</p>
    </div>
}
```

Output:



Lab no: 28 **Date: 2082/04/21**

Title: Implementation validation message and validation summary helper method in ASP .NET core MVC.

Theory:

In ASP.NET Core MVC, ValidationMessage and ValidationSummary helper methods are used to provide user-friendly validation feedback in forms. @Html.ValidationMessageFor() displays a specific error message next to a single form field when the data does not meet the validation rules, while @Html.ValidationSummary() shows all validation errors in one place, typically at the top of the form. These helpers work together with Data Annotations in the model (like [Required], [StringLength], [Range]) to ensure that user input is validated both on the client-side and server-side, improving the reliability and usability of web applications.

Controller:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Text.RegularExpressions;
using System.Web.Mvc;

namespace Lab28.Controllers
{
    public class HomeController : Controller
    {

        public ActionResult Index(string FullName, string Age, string Email)
        {
            if (Request.HttpMethod == "POST")
            {
                string emailPattern = @"^[^@\s]+@[^@\s]+\.[^@\s]+$";

                // Clear previous errors
                ModelState.Clear();

                if (string.IsNullOrWhiteSpace(FullName))
                {
                    ModelState.AddModelError("FullName", "Full Name is Required");
                    ViewData["fullNameError"] = "*";
                }

                if (string.IsNullOrWhiteSpace(Age))
                {
                    ModelState.AddModelError("Age", "Age is Required");
                    ViewData["AgeError"] = "*";
                }

                if (string.IsNullOrWhiteSpace(Email))
                {
                    ModelState.AddModelError("Email", "Email is Required");
                    ViewData["EmailError"] = "*";
                }
                else if (!Regex.IsMatch(Email, emailPattern))
                {
```

```
                    ModelState.AddModelError("Email", "Invalid Email");
                    ViewData["EmailError"] = "*";
                }

                if (ModelState.IsValid)
                {
                    ViewData["SuccessMessage"] = "<b style='color:green;'>Data
submitted successfully!</b>";
                    // Clear form data on success
                    ViewData["FullName"] = "";
                    ViewData["Age"] = "";
                    ViewData["Email"] = "";
                }
            }
            return View();
        }
    }
}
```

Index:

```
@{
    ViewBag.Title = "Index";
}

<h2>Index</h2>

@Html.ValidationSummary("", new { @class = "text-danger" })

@using (Html.BeginForm())
{
    <p>
        Enter Name: @Html.TextBox("FullName", null, new { @class = "form-control" })
        <b style="color:red;">@ViewData["fullNameError"]</b>
        @Html.ValidationMessage("FullName", "", new { @class = "text-danger" })
    </p>
    <p>
        Enter Age: @Html.TextBox("Age", null, new { @class = "form-control" })
        <b style="color:red;">@ViewData["AgeError"]</b>
        @Html.ValidationMessage("Age", "", new { @class = "text-danger" })
    </p>
    <p>
        Enter Email: @Html.TextBox("Email", null, new { @class = "form-control" })
        <b style="color:red;">@ViewData["EmailError"]</b>
        @Html.ValidationMessage("Email", "", new { @class = "text-danger" })
    </p>
    <input type="submit" value="Submit" class="btn btn-primary" />
    @Html.Raw(ViewData["SuccessMessage"])
}
```

Output:

Index - My ASP.NET Application

https://localhost:44341/Home/Index

Application name

- Home
- About
- Contact

Index

- Full Name is Required
- Age is Required
- Email is Required

Enter Name:

* Full Name is Required

Enter Age:

* Age is Required

Enter Email:

* Email is Required

Submit

© 2025 - My ASP.NET Application

Index - My ASP.NET Application

https://localhost:44341/Home/Index

Application name

- Home
- About
- Contact

Index

Enter Name:

Enter Age:

Enter Email:

Submit **Data submitted successfully!**

© 2025 - My ASP.NET Application

Lab no: 29 **Date:2082/04/28**

Title: Implementation of Data Annotation in ASP.Net MVC.

Theory:

Data Annotation in ASP.NET MVC is used to define validation rules, display formats, and metadata for model properties using attributes. It helps ensure data integrity by validating input on both client and server sides without writing complex validation logic. Common attributes include [Required], [StringLength], [Range], [EmailAddress], and [Display]

Steps:

Step 1: New Project→Asp.Net Web Application(.Net Framework)→Name(Data Annotation)→OK→Empty→MVC

Controller:

Solution Explorer→App.Start→Controller→Empty→Add→ControllerName(HomeController)

Adding View:

Index()→Right Click→Add View→View here(Index)→ Add

Adding Model:

Solution Explorer→App Start→Model→Right Click→Class→Add New Item→Data Annotation →Code

Name→ Employee.Cs→Add

Code in Employee.Cs

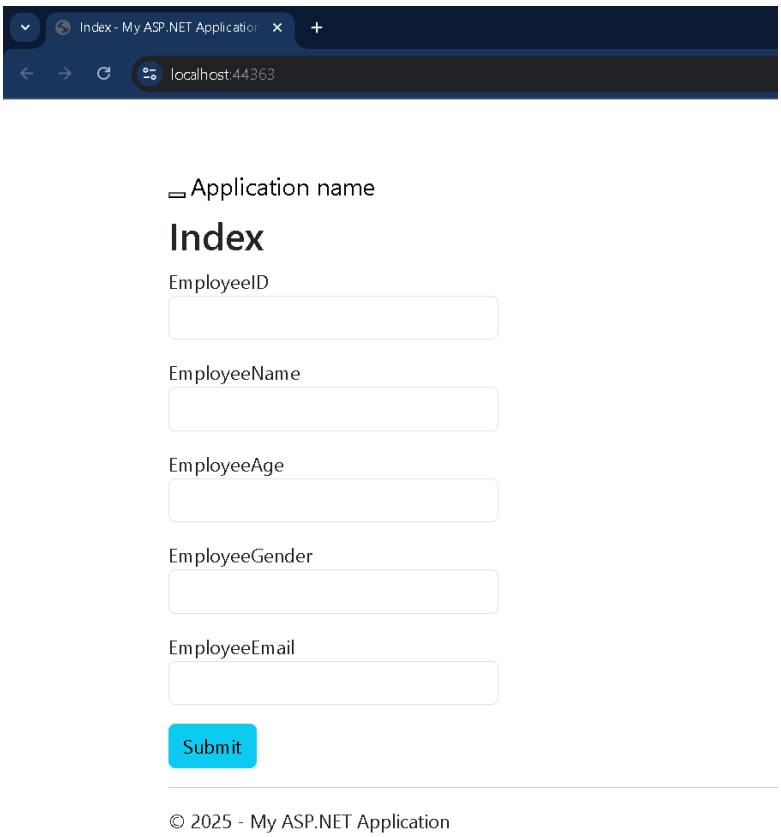
```
using System;
using System.Collections.Generic;
using System.ComponentModel.DataAnnotations;
using System.Linq;
using System.Web;

namespace DataAnnotation29.Models
{
    public class Employee
    {
        [Required(ErrorMessage ="Id is Mandatory")]
        public int EmployeeID {  get; set; }

        [Required(ErrorMessage ="Name is Mandatory")]
        public string EmployeeName { get; set; }

        [Required(ErrorMessage ="Age is Mandatory")]
        public string EmployeeDescription { get; set; }

        [Required(ErrorMessage ="Gender is Mandatory")]
        public string EmployeePhone { get; set; }

        [Required(ErrorMessage ="Email is Mandatory")]
        public int EmployeeAge { get; set; }

    }
}
```

Code in Index.cshtml

```
@model DataAnnotation29.Models.Employee
@{
    ViewBag.Title = "Index";
}

<h2>Index</h2>
@using (Html.BeginForm())
{
    <p>
        @Html.LabelFor(model => model.EmployeeID) @Html.TextBoxFor(model =>
model.EmployeeID, "", new { @class = "form-control" })
        @Html.ValidationMessageFor(model => model.EmployeeID, "", new { @class =
"text-danger" })
    </p>

    <p>
        @Html.LabelFor(model => model.EmployeeName) @Html.TextBoxFor(model =>
model.EmployeeName, "", new { @class = "form-control" })
        @Html.ValidationMessageFor(model => model.EmployeeName, "", new { @class =
"text-danger" })
    </p>

    <p>
        @Html.LabelFor(model => model.EmployeeAge) @Html.TextBoxFor(model =>
model.EmployeeAge, "", new { @class = "form-control" })
        @Html.ValidationMessageFor(model => model.EmployeeAge, "", new { @class =
"text-danger" })
    </p>

    <p>
        @Html.LabelFor(model => model.EmployeeGender) @Html.TextBoxFor(model =>
model.EmployeeGender, "", new { @class = "form-control" })
        @Html.ValidationMessageFor(model => model.EmployeeGender, "", new { @class =
"text-danger" })
    </p>

    <p>
        @Html.LabelFor(model => model.EmployeeEmail) @Html.TextBoxFor(model =>
model.EmployeeEmail, "", new { @class = "form-control" })
        @Html.ValidationMessageFor(model => model.EmployeeEmail, "", new { @class =
"text-danger" })
    </p>

    <input type="submit"value="Submit"class="btn btn-info"/>
    @Html.Raw(ViewData["SuccessMessage"])
}
```

HomeController.cs

```
using DataAnnotation29.Models;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;

namespace DataAnnotation29.Controllers
{
    public class HomeController : Controller
    {
        // GET: Home
        public ActionResult Index()
        {
            return View();
        }

        [HttpPost]
        public ActionResult Index(Employee e)
        {
            if (ModelState.IsValid == true)
            {
                ViewData["SuccessMessage"] = "<script>alert('Data has been submitted!!')</script>";
            }
            return View();
        }
    }
}
```



Lab no: 30 **Date:2082/05/05**

Title: Implementation of Attribute Based Routing in ASP.Net Core MVC 5.0.

Theory:

Attribute-Based Routing is a technique in ASP.NET Core MVC where routing information is specified directly on controller classes and action methods using attributes, rather than being configured in a centralized route table. It provides greater control and flexibility over URL patterns, especially in complex applications.

- **[Route] Attribute**: Defines the URL pattern for a controller or an action.
- **[HttpGet], [HttpPost], [HttpPut], [HttpDelete]**: Specify HTTP methods for action methods.
- **Route Parameters**: Can be defined within the route string using curly braces {}.

Steps:

Step 1:Visual Studio→New Project→Project Name(AttributeBasedRouting)

Step 2: Solution Explorer→App-Start→Controller→Right Click→Add→Controller→MVC5 Controller Empty→ Add→Student Controller→Controller(Student Controller)

Step 3: Add in RouteConfig.cs

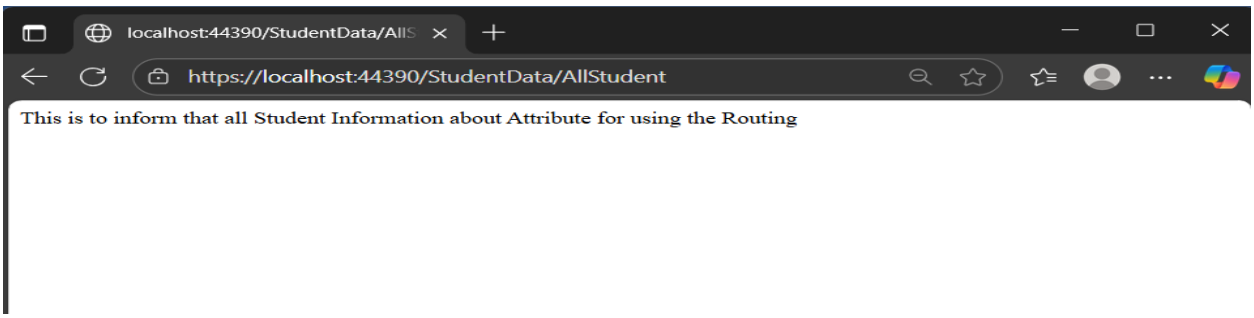
```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using System.Web.Routing;

namespace _30.AttributeBasedRouting
{
    public class RouteConfig
    {
        public static void RegisterRoutes(RouteCollection routes)
        {
            routes.IgnoreRoute("{resource}.axd/{*pathInfo}");
            routes.MapMvcAttributeRoutes(); //enable routing

            routes.MapRoute(
                name: "Default",
                url: "{controller}/{action}/{id}",
                defaults: new { controller = "Home", action = "Index", id =
UrlParameter.Optional }
            );
        }
    }
}
```

Step 4: Adding Code in StudentController.cs

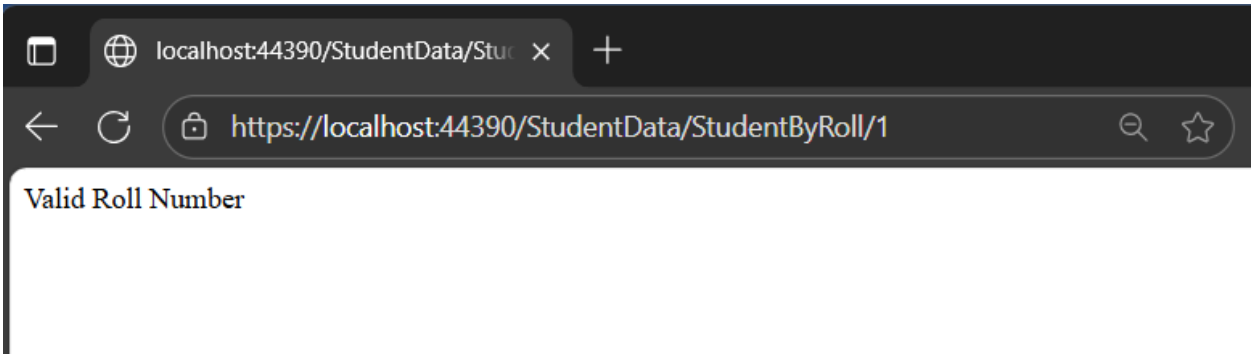
```
using System;
using System.Collections.Generic;
using System.Linq;
```

```
using System.Web;
using System.Web.Mvc;

namespace _30.AttributeBasedRouting.Controllers
{
    [RoutePrefix("StudentData")] //routeprefix
    public class StudentController : Controller
    {
        // GET: Studnet
        [Route("AllStudent")] //route students
        public ActionResult Index()
        {
            return Content("This is to inform that all Student Information about
Attribute for using the Routing");
        }
    }
}
```

Adding: localhost:44390/StudentData/AllStudent



Step 5: Add code in StudentController.cs

```
[Route("StudentByRoll/{rolln}")]
public ActionResult StudentbyRollNo(int rolln)
{
    if(rolln >0)
    {
        return Content("Valid Roll Number");
    }
    else
    {
        return Content("Invalid Roll Number");
    }
```

Adding url: localhost:44390/StudentData/StudentByRoll/1